- `policy` is the BayesOpt policy object that we will learn about shortly.
- `bounds` stores the boundary of our search space, which is -5 and 5 in this case.
- `q = 1` specifies the number of points we'd like to helper function to return, which is 1. (In Chapter 7, we will learn about the setting where we are allowed to make multiple queries to the objective functions at the same time.)
- `num_restarts` and `raw_samples` denote the number of repeats and the number of initial data points L-BFGS uses when searching for the optimal candidate that gives the highest acquisition score. Generally, I recommend using 20 times and 50 times the number of dimensions for these parameters, respectively.

SIDEBAR **Setting the number of restarts and raw samples**

The higher the values `num_restarts` and `raw_samples`, the more exhaustive L-BFGS will be when searching for the optimal candidate maximizing the acquisition score. It also means that the L-BFGS algorithm will take longer to run. You can increase these two numbers if you see that L-BFGS is failing at maximizing the acquisition score, or decrease them if the algorithm is taking too long to run.

- The returned values, `next_x` and `acq_val`, are the location of the point giving the highest acquisition score and the corresponding maximized acquisition score, respectively.

```
next_x, acq_val = botorch.optim.optimize_acqf(
    policy,
    bounds=torch.tensor([[-bound * 1.0], [bound * 1.0]]),
    q=1,
    num_restarts=20,
    raw_samples=50,
)
```

As the last step, we put together what we have implemented so far in a BayesOpt loop. At each iteration of this loop:

1. We first print out the best value that we have seen so far (`train_y.max()`), which shows how optimization is progressing.
2. Then, we retrain the GP on the current training data and redeclare the BayesOpt policy.
3. Using helper function `botorch.optim.optimize_acqf()` from BoTorch, we identify the point in the search space that maximizes the acquisition score.
4. We call helper function `visualize_gp_belief_and_policy()`, which visualizes our current GP belief and optimization progress.
5. Finally, we query the function value at the identified point (`next_x`) and update our observed data.

This entire procedure is summarized in Figure 4.4, which shows the steps in the BayesOpt loop and the corresponding code that implements them. Each step is implemented by modular code from our helper functions or BoTorch, making the entire procedure easy to follow.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>